

COP4020 Programming Languages

Liting Zhang

University of West Florida

Course Overview

- Tuesday, Thursday 11 am - 12:15 pm
- Room 438
- Course Zoom link: <https://uwf.zoom.us/j/84176259046>
- Instructor:
 - ▶ Dr. Liting Zhang
 - ▶ lzhang@uwf.edu, 4/435
 - ▶ Office hours: T, R 2 - 4 PM Via Zoom
<https://uwf.zoom.us/my/lzhang>
- TA:

Communication Policy

- Course information and updates are sent via Canvas announcements
- Email is the preferred method of communication
- Use subject line format: COP4020: [Your Topic Here]
- Example: COP4020: Project 2 Questions
- Expect a response within 1–2 business days
- Avoid:
 - ▶ Canvas/Navigation messages
 - ▶ Sending only error screenshots without your code/context

Course Goals

- Describe fundamentals issues in the design and implementation of programming languages.
- Describe programming language theory as it pertains to programming language translation
- Develop programs using the major programming paradigms (procedural, object-oriented, functional and logic-based).

Attendance Policy

- Regular attendance is expected and will greatly improve performance
- Attendance is not taken and not part of your grade
- In-class activities are graded (lowest two will be dropped)

Grading / Evaluation

Assignment	% of Grade
Participation	10%
Homeworks	10%
Projects	20%
Group Project	10%
Tests	25%
Final Exam	25%

Letter Grade	Range
A	92% – 100%
A-	90% – 91.9%
B+	87% – 89.9%
B	84% – 86.9%
B-	80% – 83.9%
C+	77% – 79.9%
C	74% – 76.9%
C-	70% – 73.9%
D+	67% – 69.9%
D	60% – 66.9%
F	0% – 59.9%

Assessments

- Individual Projects:
 - ▶ 3 programming assignments (GitHub Classroom)
 - ▶ about 2 weeks to finish
- Group Project:
 - ▶ about 5 weeks to finish
- Exams:
 - ▶ 2 tests (around week 6 and week 11)
 - ▶ 1 comprehensive final exam
 - ▶ Respondus LockDown Browser required
- Participation
 - ▶ Participation is measured using online activities including quizzes, labs and other activities.

Late Policy

- In-class activities: no late submissions
- Projects:
 - ▶ accepted up to 48 hours after deadline; after that, 0 without approved extension
 - ▶ no late submission for the last project
 - ▶ penalty (applied once to earned score):
 - ★ ≤ 24 hours late: -10%
 - ★ 24–48 hours late: -20%
 - ▶ extension: request before the deadline when possible

Excused Absences Policy

- Accepted reasons (with documentation):
 - ▶ illness, family death/crisis
 - ▶ military/jury duty, religious holy days
 - ▶ official University events
 - ▶ serious illness of dependent children
- Notify instructor ASAP for any excused absence affecting graded work
- Make-up tests:
 - ▶ allowed with medical notice or approved emergency
 - ▶ non-medical/emergency reasons require advance approval
 - ▶ if absent without prior approval, notify within 48 hours
 - ▶ make up within 2 weeks (unless long-term illness)

Today's Plan

- What do we mean by “programming language”?
- How languages sit between humans and machines
- What languages are for:
 - ▶ Communication
 - ▶ Abstraction
 - ▶ Safety and reliability
 - ▶ Performance and efficiency
- Different language styles (paradigms)
- How this connects to the rest of the course

What is a Programming Language? (Informal)

- A **system of rules** for telling a computer what to do
- Consists of
 - ▶ Syntax: what programs look like
 - ▶ Semantics: what programs mean / what they do
- Similar to human language:
 - ▶ Vocabulary, grammar, meaning
- but with stricter rules: computers don't guess what we meant

Why Not Just Use English?

- Natural languages are
 - ▶ Ambiguous: "Put the glasses on the table"
 - ▶ Context-dependent: meaning changes with situation
 - ▶ Vague: "Do it quickly" - How quick?
- Computers need:
 - ▶ Precise instructions
 - ▶ Repeatable behavior
 - ▶ No guessing / interpretation
- Programming languages = controlled, precise languages for computers

Programming Languages as Contracts

- A program is an agreement between:
 - ▶ Programmer: what they think will happen
 - ▶ Language definition / compiler (what really happens)
 - ▶ Computer (how it actually runs)
- Languages define:
 - ▶ What is legal to write
 - ▶ What a legal program must do

The Hardware Reality

- At the lowest level
 - ▶ Computers work with voltages, bits, and machine instructions
- Machine code:
 - ▶ Sequences of binary numbers (e.g., 100110000)
 - ▶ Directly control CPU operations
- Writing large programs in pure machine code:
 - ▶ Extremely error-prone
 - ▶ Hard to read, hard to change
- We need something more human-friendly

Example: Machine vs High-Level Languages

- Task: add 2 and 3, store the result
- Machine-level idea:
 - ▶ Load value 2 into a register
 - ▶ Add 3
 - ▶ Store result in memory

- High-level (e.g., python-like)

1 | $x = 2 + 3$

- Programming languages:
 - ▶ Let us think at a higher level
 - ▶ Hide a lot of hardware detail

How High-Level Code Becomes Machine Code

- A compiler: translates your program once into machine code
 - ▶ e.g., C, Rust, many others
- An interpreter:
 - ▶ Reads your program and executes it step by step
 - ▶ e.g., Python, JavaScript (simplified picture)
- Languages exist partly to make this translation possible and reliable

Layers of Abstraction

- Abstraction: ignoring details that don't matter right now
- Programming languages give you:
 - ▶ Variables instead of registers
 - ▶ Functions instead of jump instructions
 - ▶ Data structures instead of raw memory addresses
- This lets programmer:
 - ▶ Focus on problem solving
 - ▶ Not on micro-managing hardware

Core Purposes of Programming Languages

- Communication:
 - ▶ Between humans and computers
 - ▶ Between humans (reading each other's code)
- Abstraction
 - ▶ Hiding unnecessary detail
 - ▶ Organizing large systems
- Safety & reliability: preventing certain classes of bugs
- Performance & efficiency: Allowing good use of hardware resources
- Expressiveness: Letting us describe complex ideas concisely

Languages as Human-Human Communication

- Most code is read more often than written
- Language are designed for :
 - ▶ Readability: how easy to understand?
 - ▶ Maintainability: how easy to change safely?
- Style guidelines, idioms, and tooling: make languages more usable for teams
- Question: would you rather debug:
 - ▶ a clear but slow program?
 - ▶ a fast but unreadable one?

Languages as Abstraction Tools

- Languages give new building blocks:
 - ▶ Functions, classes, modules, packages
 - ▶ Types, interfaces, generics, traits
- Examples:
 - ▶ Instead of thinking in bytes, think in strings
 - ▶ Instead of loops and indices, think in map/filter/reduce
- Languages help align code structure with how we think about problems

Safety and Guarantees

- Languages can forbid dangerous actions:
 - ▶ Reading past the end of an array
 - ▶ Using uninitialized variables
- Type systems: Check consistency of data usage before running
- Memory safety, concurrency safety, null safety, etc.
- The more work the language does for us, the fewer hidden landmines in our code

Performance and Control

- Some languages aim for:
 - ▶ High performance (close to machine speed)
 - ▶ Fine-grained control over memory / CPU
- Others aim for:
 - ▶ Productivity: fast to write, easier to change
 - ▶ Automatic memory management (garbage collection)
- Trade-off: Control vs convenience
- Different domains have different priorities

Language Roles in Practice

- Embedded systems: tight performance and memory constraints
- Web backends: serving requests, talking to databases
- Front-end / UI: running in browsers or mobile apps
- Data / ML: cleaning, analyzing, training models
- DevOps / scripting: automating repetitive tasks
- Different roles emphasize different language features

Domain-Specific Languages

- Shader languages (graphics / GPUD):
 - ▶ GLSL - OpenGL shading language
 - ▶ HLSL – High Level Shading Language (DirectX)
 - ▶ MSL – Metal Shading Language (Apple platforms)
 - ▶ Used to program how pixels/vertices are drawn on screen
- Configuration languages:
 - ▶ YAML, JSON, TOML - app & service config files
 - ▶ Focus on declaring setting, not computation
- Scripting languages:
 - ▶ Bash / shell scripts – automating command-line tasks
 - ▶ Python, Ruby, Perl – glue code, automation, tools
 - ▶ JavaScript – scripting in browsers and many tools
 - ▶ Often used to automate or “glue together” other systems
- Other domain-specific examples:
 - ▶ SQL – querying and updating relational databases
 - ▶ HTML/CSS – describing structure and style of web pages
- Why create a new language?
 - ▶ Make certain tasks easier, safer, less repetitive
 - ▶ Capture expert patterns in the language itself

Quick Demos: JavaScript & Python

- **Python** (general-purpose, data, scripts, ML):

```
1 | nums = [1, 2, 3, 4]
2 | total = sum(nums)
3 | print("Total:", total)
```

- **JavaScript** (runs in browsers, also on servers with Node.js):

```
1 | const nums = [1, 2, 3, 4];
2 | const total = nums.reduce((a, b) => a + b, 0);
3 | console.log("Total:", total);
```

Language Cameos in Industry

- **C++:**
 - ▶ Systems programming, game engines, performance-critical code
- **Java / C#:**
 - ▶ Enterprise backends, Android (Java), desktop apps, large systems
- **SQL:**
 - ▶ Querying and updating relational databases
- **Go:**
 - ▶ Cloud services, infrastructure tools, networking
- **Rust:**
 - ▶ Systems programming with strong safety guarantees

Paradigms: Different Ways of Thinking

- A paradigm is a style / philosophy of programming
 - ▶ Imperative: change state step by step
 - ▶ Object-Oriented: organize code around objects & methods
 - ▶ Functional: emphasize functions, immutability
 - ▶ Logic/Declarative: state what should be true, not how
- Many languages support multiple paradigms
- Paradigms influence:
 - ▶ How we decompose problems
 - ▶ The kinds of bugs we're likely to make

Same Problem, Different Styles

- Example: Sum of a list

- Imperative style:

```
1 | total = 0
2 | for x in nums:
3 |     total += x
```

- Functional style:

```
1 | total = sum(nums)
```

- Declarative (SQL-ish idea)

```
1 | SELECT SUM(value) FROM numbers;
```

Languages Shape How We Think

- Languages encourage some patterns, discourage others
 - ▶ Easy recursion? -ı You think recursively more often
 - ▶ Easy objects? -ı You think in terms of entities & messages
- Learning new languages:
 - ▶ Expands your mental toolbox
 - ▶ Changes what feels "natural" when solving problems
- This course: helps you see past any one language

Languages and Jobs

- You might see:
 - ▶ “Python / SQL” in data and analytics roles
 - ▶ “Java / C#” in enterprise and backend roles
 - ▶ “JavaScript / TypeScript” in web front-end roles
 - ▶ “C / C++ / Rust” in systems, embedded, game engines
 - ▶ “Go” in cloud / infrastructure / DevOps tools
- This course:
 - ▶ Focuses on **concepts** that transfer across languages
 - ▶ Helps you pick up new languages faster on the job

Summary: What Are Languages Really For?

- Bridging humans and machines. Making it possible to express intent precisely
- Managing complexity: abstractions, modularity, organization
- Enforcing constraints: catching errors early, improving reliability
- Optimizing for resources: time, memory, energy
- Shaping thought: Providing concepts that guide how we solve problems